CS 555 — Numerical Methods for PDEs

Homework Set 4:

Advection–Diffusion with Linear FEM and BDF3/EXT3

Atharva Deshpande

April 15, 2026

1 Problem Statement

We solve the one-dimensional advection–diffusion equation on $\Omega = [0, L]$:

$$u_t + c\,u_x \;=\; \nu\,u_{xx} + f, \qquad u(0,t) = u(L,t) = 0, \tag{1}$$

with $u(x,0) = u^0(x)$, using piecewise-linear (P1) finite elements in space and BDFk/EXTk ($k = 1, 2, 3$) time integration. Unless otherwise stated, $L = 1$, $c = 1$, $\nu = 10^{-3}$, and $f = 1$.

Problem 1 (Steady state, $u_t = 0$):

(a) Uniform mesh with $E = 100$ elements.
(b) Geometric mesh with $E = 20$, ratio $s = 0.7$.
(c) Find the optimal s for $E = 20$ by trial and error.

Problem 2 (Unsteady, $f = 0$, $u(0,t) = \sin(\pi t)$, $u^0 = 0$):

(a) Dirichlet outflow: $u(1,t) = 0$. Plot at $t = 1.5$.
(b) Neumann outflow: $\partial u/\partial x\big|_{x=L} = 0$. Plot at $t = 1.5$. Compare with (a).

2 Methodology

2.1 FEM Assembly via the Scatter Operator

Following the approach in the course notes, the global Neumann-form matrices $\bar{A}$ (stiffness), $\bar{B}$ (mass), and $\bar{C}$ (advection) are assembled using the scatter operator Q:

$$\bar{A} = Q^T A_L\, Q, \qquad \bar{B} = Q^T B_L\, Q, \qquad \bar{C} = Q^T C_L\, Q, \tag{2}$$

where A_L, B_L, C_L are block-diagonal matrices of local element contributions. For a linear element Ω^e with length $L_e = x_e - x_{e-1}$, the 2×2 local matrices are:

$$A^e = \frac{1}{L_e}\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \quad B^e = \frac{L_e}{6}\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \quad C^e = \frac{c}{2}\begin{bmatrix} -1 & 1 \\ -1 & 1 \end{bmatrix}. \tag{3}$$

Note that C^e is independent of L_e; the $1/L_e$ from the derivative of the basis function cancels the $L_e/2$ Jacobian from the element mapping $x^e(r) = x_{e-1} + \frac{L_e}{2}(r+1)$.

2.2 Boundary Treatment: Restriction and Prolongation

Boundary conditions are enforced using the restriction operator R and prolongation operator R^T. The solution is decomposed as

$$\underline{u} \;=\; R^T\,\underline{u}_0 \;+\; \underline{u}_b, \tag{4}$$

where $\underline{u}_0 \in \mathbb{R}^n$ contains the unknown interior DOFs and $\underline{u}_b$ holds the prescribed boundary values.

For Dirichlet conditions at both ends, $R^T \in \mathbb{R}^{(N+1)\times(N-1)}$ is the identity padded with zero rows at the boundary nodes. For the Dirichlet–Neumann case (Problem 2b), only the left boundary row is zeroed, and the right node remains an active DOF.

2.3 Steady Solver (Problem 1)

Setting $u_t = 0$ and $u(0) = u(1) = 0$ in the weak form gives the restricted linear system:

$$(\nu\,A + C)\,\underline{u}_0 \;=\; R\,\bar{B}\,\mathbf{1}\,f, \tag{5}$$

where $A = R\bar{A}R^T$ and $C = R\bar{C}R^T$ are the restricted stiffness and advection operators. The exact solution used for error measurement is

$$u(x) = \frac{f}{c}\,x - \frac{fL}{c}\,\frac{\exp\!\big(c(x-L)/\nu\big) - \exp(-cL/\nu)}{1-\exp(-cL/\nu)}\,. \tag{6}$$

2.4 Geometric Mesh

The geometric mesh has element lengths satisfying $L_e = s\,L_{e-1}$ for $e = 2,\ldots,E$. The first element length is

$$L_1 = L\,\frac{1-s}{1-s^E}\,. \tag{7}$$

When $s < 1$, elements shrink toward $x = L$, clustering nodes in the boundary layer.

2.5 Unsteady Solver: BDF3/EXT3 with IMEX Splitting (Problem 2)

The semi-discrete ODE is

$$\bar{B}\,\underline{u}_t \;=\; -\bar{C}\,\underline{u} \;-\; \nu\,\bar{A}\,\underline{u}. \tag{8}$$

We apply an IMEX splitting: diffusion ($-\nu\bar{A}\underline{u}$) is treated implicitly, while advection ($-\bar{C}\underline{u}$) is extrapolated explicitly via EXTk. The BDFk time derivative gives the left-hand side.

For BDF3/EXT3 at full order ($k = 3$), the update reads

$$\bar{H}\,\underline{u}^l \;=\; \frac{1}{\Delta t}\,\bar{B}\left(3\underline{u}^{l-1} - \tfrac{3}{2}\underline{u}^{l-2} + \tfrac{1}{3}\underline{u}^{l-3}\right) \;-\; \bar{C}\left(3\underline{u}^{l-1} - 3\underline{u}^{l-2} + \underline{u}^{l-3}\right), \tag{9}$$

where the Helmholtz operator is $\bar{H} = \frac{\beta_0}{\Delta t}\,\bar{B} + \nu\,\bar{A}$ with $\beta_0 = 11/6$. Boundary lifting subtracts $\bar{H}\,\underline{u}_b$ from the right-hand side, and the restricted system $H\,\underline{u}_0 = R(\cdots)$ is solved at each step.

Startup uses BDF1/EXT1 for step 1 and BDF2/EXT2 for step 2. Parameters: $E = 100$, $\Delta t = 10^{-3}$ (CFL $= c\Delta t/h = 0.1$), integrated to $t = 1.5$ (1500 steps).

3 Results

3.1 Problem 1: Steady-State Solutions

Table 1 summarizes the error metrics for all three mesh configurations.

Table 1: Problem 1: Error summary for the steady advection–diffusion problem.

Part	Mesh type	E	Max relative error	Max absolute error
(a)	Uniform	100	6.735×10^{-1}	6.667×10^{-1}
(b)	Geometric, $s = 0.70$	20	1.197×10^{-2}	1.058×10^{-2}
(c)	Geometric, $s = 0.65$	20	8.819×10^{-3}	8.168×10^{-3}

Q1(a): Uniform Mesh, $E = 100$

The uniform mesh fails to resolve the sharp boundary layer near $x = 1$. With $\nu = 10^{-3}$ and $c = 1$, the boundary-layer thickness is $\delta \sim \nu/c = 10^{-3}$, but the element size is $h = 0.01$, giving a cell Péclet number $\mathrm{Pe}_h = ch/(2\nu) = 5 \gg 1$. The centered Galerkin scheme produces large spurious oscillations near the outflow, with a maximum pointwise relative error of 6.735×10^{-1}.

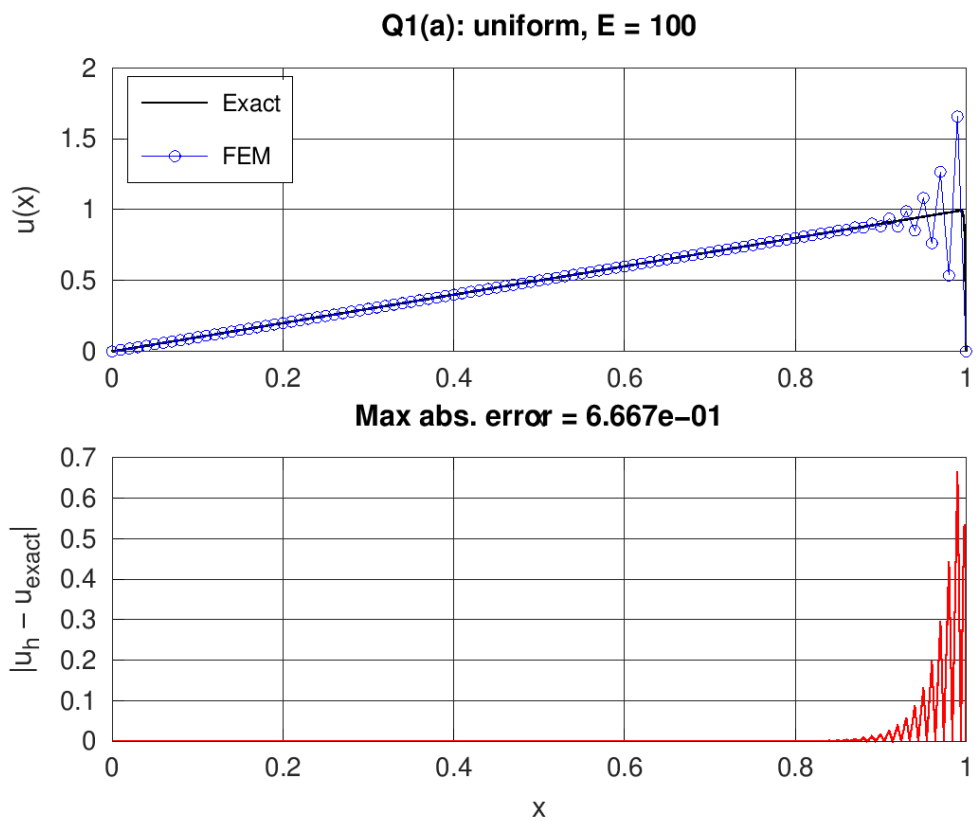

Figure 1: Q1(a): Uniform mesh with $E = 100$. Top: FEM vs. exact solution. Bottom: pointwise absolute error. Large oscillations appear near the boundary layer at $x = 1$.

Q1(b): Geometric Mesh, $E = 20$, $s = 0.70$

Using a geometric mesh with $s = 0.7$ clusters nodes near $x = 1$, placing progressively smaller elements where the boundary layer resides. Despite using only 20 elements (vs. 100 uniform), the maximum relative error drops to 1.197×10^{-2} — a factor of ~ 56 improvement. The oscillations are virtually eliminated.

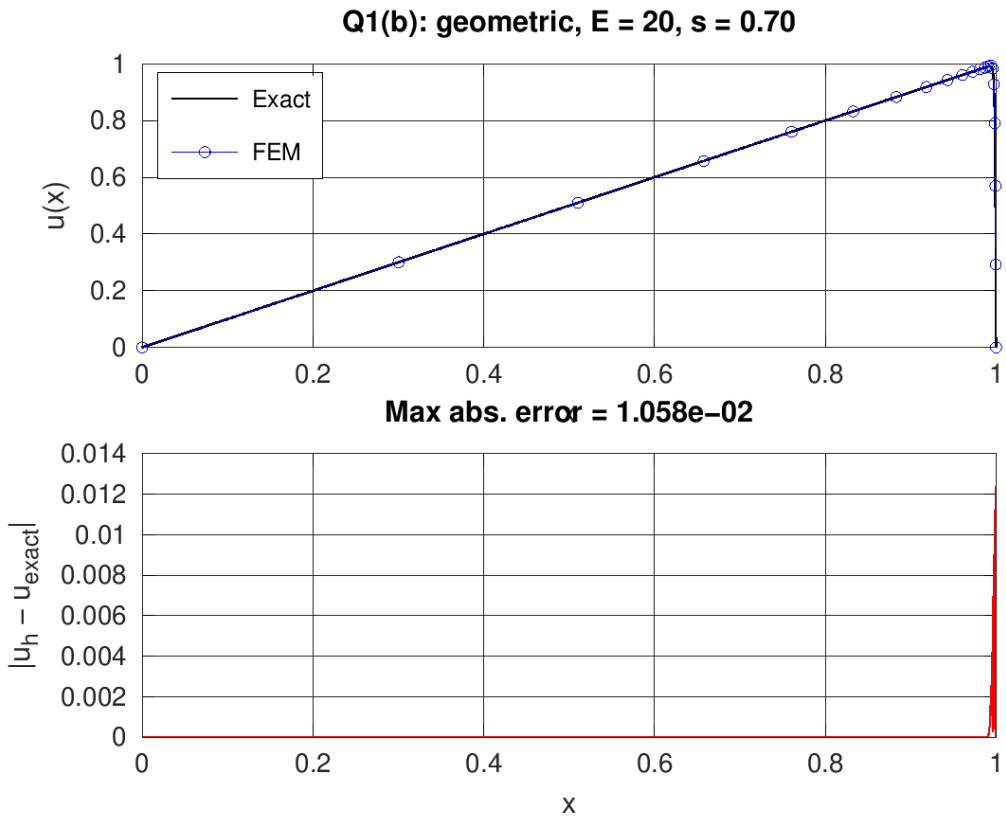

Figure 2: Q1(b): Geometric mesh with $E = 20$ and $s = 0.70$. The clustered mesh resolves the boundary layer, reducing error by nearly two orders of magnitude compared to the uniform case.

Q1(c): Optimal Geometric Ratio for $E = 20$

A sweep over $s \in [0.50, 1.05]$ with $\Delta s = 0.005$ (111 trial values) was performed, using maximum pointwise relative error as the optimization metric. The optimal value found is

$$s^* = 0.650, \qquad \text{max relative error} = 8.819 \times 10^{-3}.$$

This is a further $\sim 26\%$ reduction compared to $s = 0.70$. Figure 4 shows the error landscape: the curve has a clear minimum near $s \approx 0.65$ and grows rapidly for $s \to 1$ (approaching a uniform mesh) and more gently for $s \to 0.5$ (over-clustering at $x = 1$ at the expense of resolution in the interior).

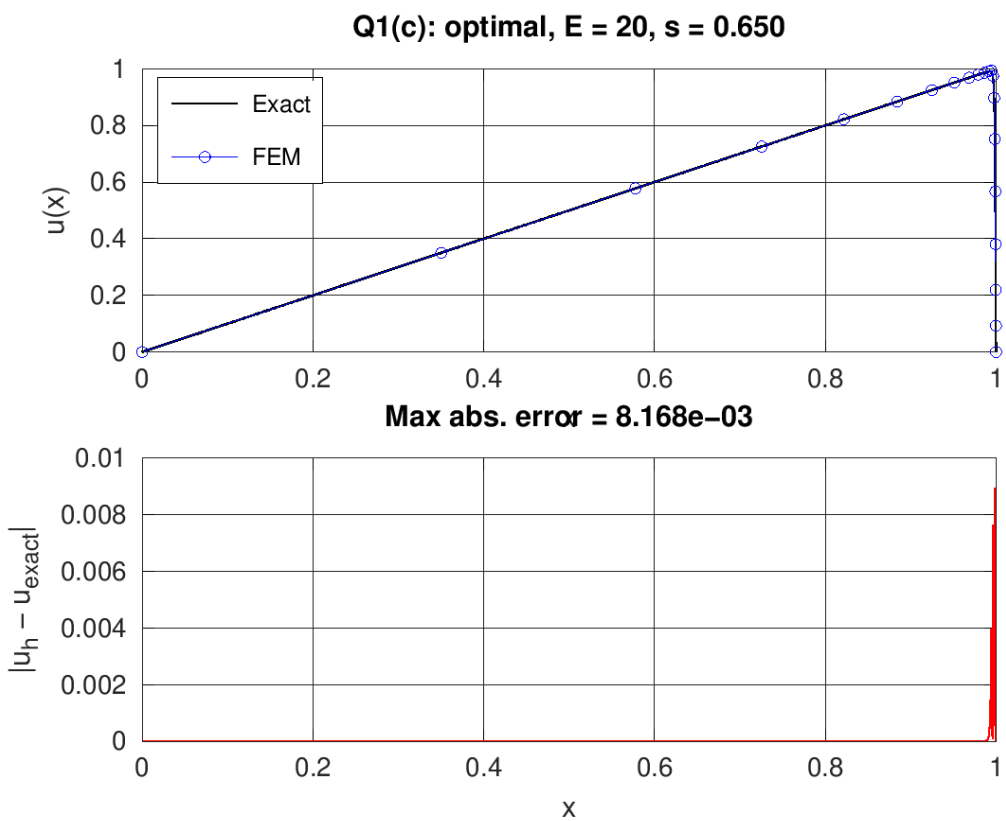

Figure 3: Q1(c): Best geometric mesh found ($s = 0.650$, $E = 20$). The FEM solution closely tracks the exact solution, with error confined to a narrow region around $x = 1$.

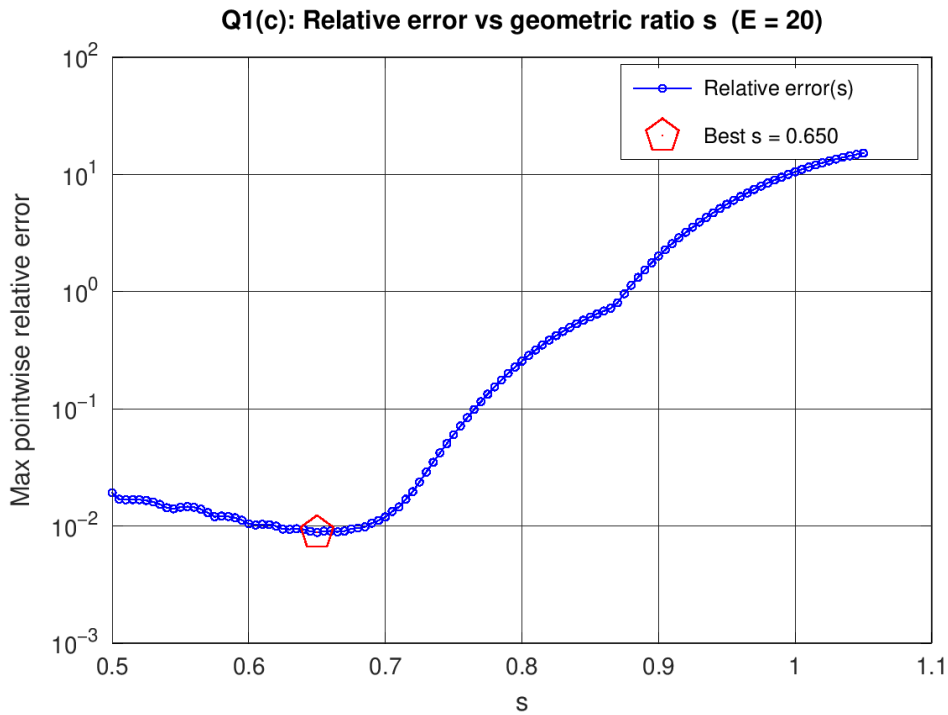

Figure 4: Q1(c): Maximum pointwise relative error as a function of the geometric ratio s for $E = 20$. The optimum lies near $s = 0.65$. For $s > 0.75$, the error grows rapidly as the mesh under-resolves the boundary layer.

3.2 Problem 2: Unsteady Solutions at $t = 1.5$

Parameters: $L = 1$, $c = 1$, $\nu = 10^{-3}$, $f = 0$, $E = 100$, $\Delta t = 10^{-3}$, $u(0, t) = \sin(\pi t)$, $u^0 = 0$.

Table 2: Problem 2: Solution statistics at $t = 1.5$.

Case	$\min(u)$	$\max(u)$	$u(1,\ 1.5)$
(a) Dirichlet: $u(1, t) = 0$	-1.000	$+1.650$	0.000
(b) Neumann: $\partial u/\partial x(1, t) = 0$	-1.000	$+0.990$	$+0.990$

Q2(a): Dirichlet Outflow, $u(1, t) = 0$

At $t = 1.5$, $\sin(\pi{\cdot}1.5) = -1$, so the inflow drives $u(0) = -1$. Material advected from the left has been accumulating positive values from earlier phases of the sine wave. Forcing $u(1, t) = 0$ at the outflow creates a conflict: the interior solution wants to be ~ 1 near $x = 1$, but the boundary condition pins it to zero. This mismatch produces a thin numerical boundary layer, and because $\mathrm{Pe}_h = 5 > 1$ on the uniform mesh, the unstabilized Galerkin scheme generates strong oscillations. In the last 20 elements, the slope changes sign 13 times, and the solution overshoots to $\max(u) = 1.650$.

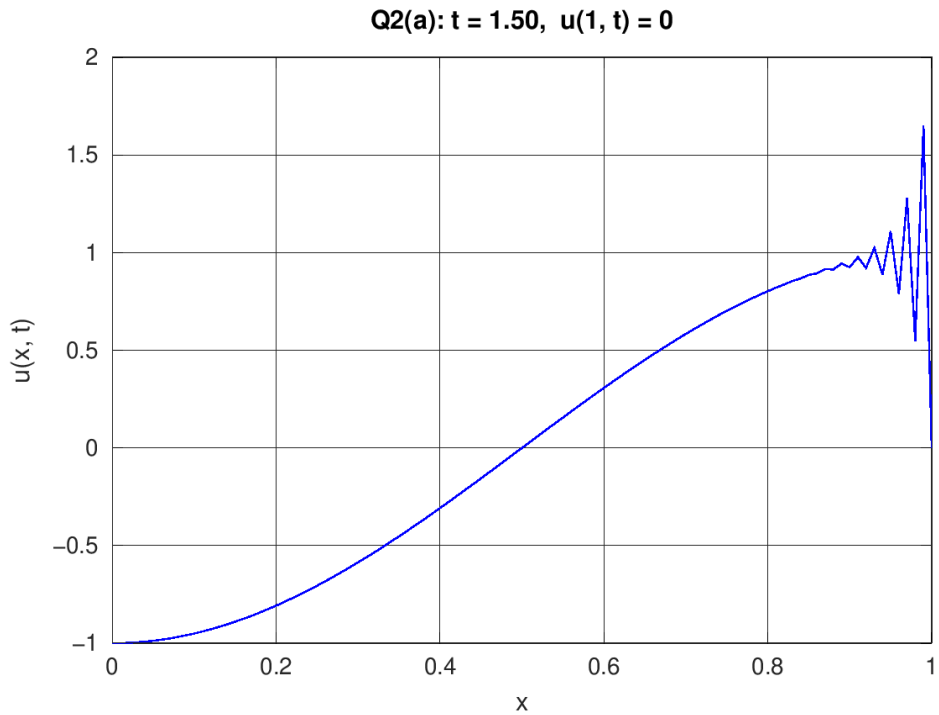

Figure 5: Q2(a): Solution at $t = 1.5$ with Dirichlet outflow $u(1, t) = 0$. Strong spurious oscillations appear near $x = 1$ due to the incompatibility of the boundary condition with the advected state.

Q2(b): Neumann Outflow, $\partial u/\partial x(1, t) = 0$

Replacing the right Dirichlet condition with the natural (homogeneous Neumann) condition $\partial u/\partial x\big|_{x=1} = 0$ removes the forced outflow constraint. In the FEM formulation, this simply means keeping the right boundary node as an active DOF in the restriction operator R (no row is removed for node N). The resulting profile at $t = 1.5$ is smooth and nearly monotone, with $u(1, 1.5) \approx 0.990$ and zero slope reversals near the outflow.

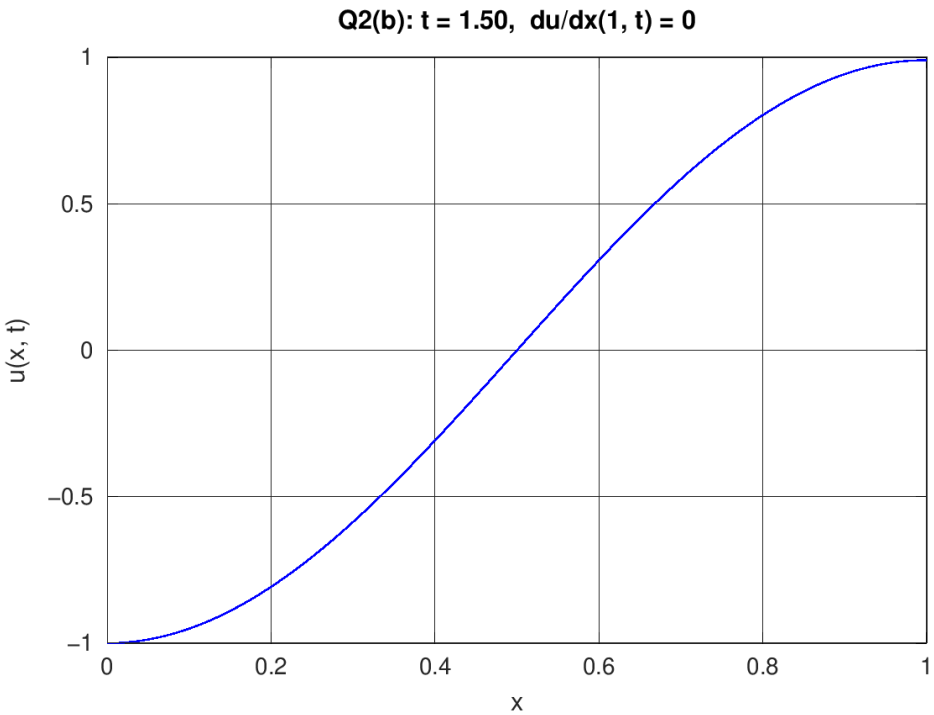

Figure 6: Q2(b): Solution at $t = 1.5$ with Neumann outflow $\partial u/\partial x(1, t) = 0$. The profile is smooth, with no spurious oscillations at the right boundary.

Comparison of Q2(a) and Q2(b)

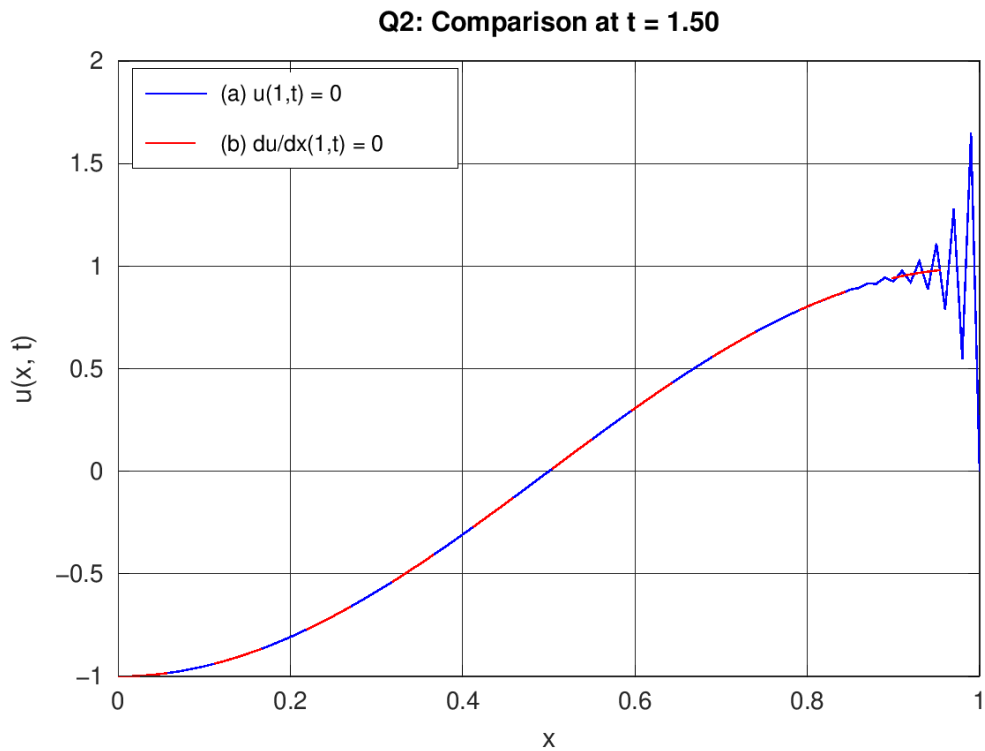

Figure 7: Comparison of Q2(a) and Q2(b) at $t = 1.5$. The two solutions agree in the interior ($x \lesssim 0.85$) but diverge sharply near the right boundary.

The key differences are:

- **Oscillations:** Part (a) exhibits 13 slope reversals in the last 20 elements and a maximum overshoot of $u \approx 1.65$. Part (b) has zero slope reversals and a bounded maximum of $u \approx 0.99$.
- **Physical interpretation:** The Dirichlet condition $u(1, t) = 0$ is incompatible with the interior advected state and acts as an artificial reflector, creating an outflow boundary layer. The Neumann condition acts as a transparent (or "do-nothing") outflow, allowing material to exit the domain naturally.
- **Interior agreement:** Both solutions match closely for $x \lesssim 0.85$, confirming that the boundary treatment only affects the near-outflow region.
- **Péclet number:** With $h = 0.01$ and $\nu = 10^{-3}$, $\mathrm{Pe}_h = ch/(2\nu) = 5$. In the absence of stabilization (e.g., SUPG), the centered Galerkin method oscillates whenever $\mathrm{Pe}_h > 1$ and the boundary condition creates a sharp gradient.

4 Implementation Details

The complete implementation is a single MATLAB/Octave file `hw3.m`. The code is organized into modular functions:

- `assemble_fem` — Builds $\bar{A}$, $\bar{B}$, $\bar{C}$ via the sparse scatter operator Q.
- `geometric_mesh` — Generates a geometric progression of element sizes with ratio s.

- `exact_steady` — Evaluates the closed-form steady solution (6), written in a numerically stable form that avoids overflow from $\exp(cL/\nu)$.
- `solve_steady` — Assembles and solves the restricted steady system.
- `bdf3ext3` — Time-steps the unsteady problem with BDFk/EXTk and boundary lifting.
- `build_restriction` — Constructs R for Dirichlet–Dirichlet or Dirichlet–Neumann configurations.

5 Source Code

5.1 Main Driver and Assembly

```
function hw3()
% CS555 HW4: Advection-Diffusion with Linear FEM + BDF3/EXT3
%
%   u_t + c u_x = nu u_xx + f    on [0, L]
%
% Q1: Steady state (u_t=0), f=1, u(0)=u(1)=0
% Q2: Unsteady,     f=0,    u(0,t)=sin(pi*t)

  if any(strcmp(available_graphics_toolkits(), 'gnuplot'))
    graphics_toolkit('gnuplot');
  end
  set(0, 'defaultfigurevisible', 'off');

  L = 1;  c = 1;  nu = 1e-3;  f = 1;

  %% =============== PROBLEM 1: STEADY STATE ===============
  % --- Q1(a): uniform mesh, E = 100 ---
  E = 100;
  x = linspace(0, L, E+1)';
  [u_a, ue_a] = solve_steady(x, c, nu, f, L);
  [abs_a, rel_a] = compute_error(u_a, ue_a);
  fprintf('Q1(a): E = %d, uniform mesh\n', E);
  fprintf('  Max pointwise relative error: %.6e\n', rel_a);
  fprintf('  Max absolute error:           %.6e\n\n', abs_a);
  plot_q1(x, u_a, c, nu, f, L, abs_a, ...
          'Q1(a): uniform, E = 100', 'q1a_uniform.png');

  % --- Q1(b): geometric mesh, E = 20, s = 0.7 ---
  E = 20;  s = 0.7;
  x = geometric_mesh(L, E, s);
  [u_b, ue_b] = solve_steady(x, c, nu, f, L);
  [abs_b, rel_b] = compute_error(u_b, ue_b);
  fprintf('Q1(b): E = %d, s = %.2f, geometric mesh\n', E, s);
  fprintf('  Max pointwise relative error: %.6e\n', rel_b);
  fprintf('  Max absolute error:           %.6e\n\n', abs_b);
  plot_q1(x, u_b, c, nu, f, L, abs_b, ...
          sprintf('Q1(b): geometric, E = %d, s = %.2f', E, s), ...
          'q1b_geometric_s0p7.png');

  % --- Q1(c): find optimal s for E = 20 ---
  E = 20;
  s_vals = 0.50:0.005:1.05;
  abs_vs_s = zeros(size(s_vals));
  rel_vs_s = zeros(size(s_vals));
  for k = 1:length(s_vals)
    xk = geometric_mesh(L, E, s_vals(k));
```

```
    [uk, uek] = solve_steady(xk, c, nu, f, L);
    [abs_vs_s(k), rel_vs_s(k)] = compute_error(uk, uek);
  end
  [best_rel, idx] = min(rel_vs_s);
  s_opt = s_vals(idx);

  x = geometric_mesh(L, E, s_opt);
  [u_c, ue_c] = solve_steady(x, c, nu, f, L);
  [abs_c, rel_c] = compute_error(u_c, ue_c);
  plot_q1(x, u_c, c, nu, f, L, abs_c, ...
          sprintf('Q1(c): optimal, E = %d, s = %.3f', E, s_opt), ...
          'q1c_best_s.png');

  % Error-vs-s plot
  fig = figure('visible', 'off');
  semilogy(s_vals, rel_vs_s, 'b-o', 'linewidth', 1.5, 'markersize', 3);
  hold on;
  semilogy(s_opt, best_rel, 'rp', 'markersize', 14, 'linewidth', 2);
  hold off;
  xlabel('s');  ylabel('Max pointwise relative error');
  title('Q1(c): Relative error vs geometric ratio s  (E = 20)');
  legend('Relative error(s)', sprintf('Best s = %.3f', s_opt), ...
         'location', 'northeast');
  grid on;
  print(fig, 'q1c_error_vs_s.png', '-dpng', '-r200');
  close(fig);

  %% =============== PROBLEM 2: UNSTEADY ===============
  E  = 100;
  x  = linspace(0, L, E+1)';
  n  = E + 1;
  dt = 1e-3;                    % CFL = c*dt/h = 0.1
  t_end  = 1.5;
  nsteps = round(t_end / dt);
  dt = t_end / nsteps;

  [Bbar, Cbar, Abar] = assemble_fem(x, c);

  % --- Q2(a): Dirichlet both ends ---
  ua = bdf3ext3(Bbar, Cbar, Abar, n, dt, nsteps, nu, ...
                @(t) sin(pi*t), true, @(t) 0);

  fig = figure('visible', 'off');
  plot(x, ua, 'b-', 'linewidth', 2);
  xlabel('x');  ylabel('u(x, t)');
  title(sprintf('Q2(a): t = %.2f,  u(1, t) = 0', t_end));
  grid on;
  print(fig, 'q2a_t1p5.png', '-dpng', '-r200');
  close(fig);

  % --- Q2(b): Left Dirichlet, Right Neumann ---
  ub = bdf3ext3(Bbar, Cbar, Abar, n, dt, nsteps, nu, ...
                @(t) sin(pi*t), false, []);

  fig = figure('visible', 'off');
  plot(x, ub, 'b-', 'linewidth', 2);
  xlabel('x');  ylabel('u(x, t)');
  title(sprintf('Q2(b): t = %.2f,  du/dx(1, t) = 0', t_end));
  grid on;
```

```
  print(fig, 'q2b_t1p5.png', '-dpng', '-r200');
  close(fig);

  % Comparison plot
  fig = figure('visible', 'off');
  plot(x, ua, 'b-', 'linewidth', 2);  hold on;
  plot(x, ub, 'r--', 'linewidth', 2); hold off;
  xlabel('x');  ylabel('u(x, t)');
  title(sprintf('Q2: Comparison at t = %.2f', t_end));
  legend('(a) u(1,t) = 0', '(b) du/dx(1,t) = 0', 'location', 'northwest');
  grid on;
  print(fig, 'q2_comparison.png', '-dpng', '-r200');
  close(fig);
end

%%  ASSEMBLY:  Abar = Q' * A_L * Q   (and similarly B, C)
function [Bbar, Cbar, Abar] = assemble_fem(xnodes, c)
  E  = length(xnodes) - 1;
  n  = E + 1;
  nL = 2 * E;

  % scatter operator Q  (nL x n, sparse Boolean)
  Qi = zeros(nL, 1);   Qj = zeros(nL, 1);
  for e = 1:E
    Qi(2*e - 1) = 2*e - 1;   Qj(2*e - 1) = e;
    Qi(2*e)     = 2*e;       Qj(2*e)     = e + 1;
  end
  Q = sparse(Qi, Qj, 1, nL, n);

  % block-diagonal local matrices  (nL x nL)
  ntriplets = 4 * E;
  ri = zeros(ntriplets, 1);  ci = zeros(ntriplets, 1);
  av = zeros(ntriplets, 1);  bv = zeros(ntriplets, 1);
  cv = zeros(ntriplets, 1);

  k = 0;
  for e = 1:E
    Le = xnodes(e + 1) - xnodes(e);
    Ae = (1 / Le) * [1, -1; -1, 1];
    Be = (Le / 6) * [2,  1;  1, 2];
    Ce = (c / 2)  * [-1, 1; -1, 1];

    base = 2 * (e - 1);
    for p = 1:2
      for q = 1:2
        k = k + 1;
        ri(k) = base + p;   ci(k) = base + q;
        av(k) = Ae(p, q);   bv(k) = Be(p, q);
        cv(k) = Ce(p, q);
      end
    end
  end

  AL = sparse(ri, ci, av, nL, nL);
  BL = sparse(ri, ci, bv, nL, nL);
  CL = sparse(ri, ci, cv, nL, nL);

  Abar = Q' * AL * Q;
  Bbar = Q' * BL * Q;
```

```
  Cbar = Q' * CL * Q;
end
```

Listing 1: Main function and FEM assembly (`hw3.m`, lines 1–209).

5.2 Geometric Mesh and Exact Solution

```
function x = geometric_mesh(L, E, s)
  if abs(s - 1) < 1e-14
    x = linspace(0, L, E + 1)';
  else
    L1 = L * (1 - s) / (1 - s^E);
    Le = L1 * s .^ (0:E-1)';
    x  = [0; cumsum(Le)];
    x(end) = L;                        % enforce exact endpoint
  end
end

function u = exact_steady(x, c, nu, f, L)
  eta = exp(-c * L / nu);            % ~ 0 when cL/nu >> 1
  u   = (f / c) * x ...
      - (f * L / c) * (exp(c * (x - L) / nu) - eta) ./ (1 - eta);
end
```

Listing 2: Geometric mesh and exact steady solution.

5.3 Steady Solver and Error Computation

```
function [u, ue] = solve_steady(x, c, nu, f, L)
  [Bbar, Cbar, Abar] = assemble_fem(x, c);
  n = length(x);

  R = build_restriction(n, true, true);
  Rt = R';

  A = R * Abar * Rt;
  C = R * Cbar * Rt;
  fbar = f * Bbar * ones(n, 1);

  u_int = (nu * A + C) \ (R * fbar);
  u = Rt * u_int;
  ue = exact_steady(x, c, nu, f, L);
end

function [max_abs, max_rel] = compute_error(u, ue)
  max_abs = max(abs(u - ue));
  n = length(u);
  ii = 2:n-1;
  nz = abs(ue(ii)) > 1e-15;
  if any(nz)
    max_rel = max(abs(u(ii(nz)) - ue(ii(nz))) ./ abs(ue(ii(nz))));
  else
    max_rel = max_abs;
  end
end
```

Listing 3: Steady solver and error metrics.

5.4 BDF3/EXT3 Time Stepper

```
function u_final = bdf3ext3(Bbar, Cbar, Abar, n, dt, nsteps, nu, ...
                            u_left_fn, right_dirichlet, u_right_fn)
  b0 = [1, 3/2, 11/6];

  R = build_restriction(n, true, right_dirichlet);
  Rt = R';

  % Pre-build Helmholtz matrices for each BDF order
  Hbar = cell(3, 1);
  H = cell(3, 1);
  for ord = 1:3
    Hbar{ord} = (b0(ord) / dt) * Bbar + nu * Abar;
    H{ord} = R * Hbar{ord} * Rt;
  end

  % Solution history
  uh = { zeros(n, 1),  zeros(n, 1),  zeros(n, 1) };

  for step = 1:nsteps
    t_new = step * dt;

    if step == 1           % BDF1/EXT1
      bdf_rhs = uh{1};
      ext_rhs = uh{1};
      ord = 1;
    elseif step == 2       % BDF2/EXT2
      bdf_rhs = 2 * uh{1} - 0.5 * uh{2};
      ext_rhs = 2 * uh{1} -       uh{2};
      ord = 2;
    else                   % BDF3/EXT3
      bdf_rhs = 3 * uh{1} - 1.5 * uh{2} + (1/3) * uh{3};
      ext_rhs = 3 * uh{1} - 3   * uh{2} +         uh{3};
      ord = 3;
    end

    ub = zeros(n, 1);
    ub(1) = u_left_fn(t_new);
    if right_dirichlet
      ub(n) = u_right_fn(t_new);
    end

    rhs = (1/dt)*(Bbar*bdf_rhs) - Cbar*ext_rhs - Hbar{ord}*ub;
    u0_new = H{ord} \ (R * rhs);
    u_new = Rt * u0_new + ub;

    uh{3} = uh{2};
    uh{2} = uh{1};
    uh{1} = u_new;
  end

  u_final = uh{1};
end
```

Listing 4: BDF3/EXT3 unsteady solver with boundary lifting.

5.5 Restriction Operator

```
function R = build_restriction(n, left_dirichlet, right_dirichlet)
  active = true(1, n);
  if left_dirichlet
    active(1) = false;
  end
  if right_dirichlet
    active(n) = false;
  end
  free = find(active);
  R = sparse(1:length(free), free, 1, length(free), n);
end
```

Listing 5: Boundary restriction operator.

6 Conclusions

1. **Mesh refinement matters more than element count.** A geometric mesh with only $E = 20$ elements and $s = 0.65$ achieves a maximum relative error of 8.8×10^{-3}, while a uniform mesh with $E = 100$ elements yields an error of 6.7×10^{-1}. Intelligent node placement that respects the boundary-layer structure of the solution is far more effective than brute-force uniform refinement.

2. **Outflow boundary conditions strongly affect solution quality.** For advection-dominated problems, an incompatible Dirichlet outflow condition creates a numerical boundary layer that the unstabilized Galerkin scheme cannot resolve, producing large oscillations. A natural Neumann outflow condition avoids this issue entirely, yielding a smooth, bounded solution.

3. **The BDF3/EXT3 IMEX scheme is effective and stable.** With $\Delta t = 10^{-3}$ and CFL $= 0.1$, the scheme runs stably for 1500 steps to $t = 1.5$. The startup procedure (BDF1→BDF2→BDF3) avoids introducing spurious transients.